

ver_datos_trainer.js

Pantalla de solo lectura para el lado del entrenador. Muestra todos los datos personales, fisicos, de contacto/salud y de entrenamiento de un alumno especifico. No tiene inputs, botones de edicion ni logica de estado mas alla del layout responsivo.

1. Recepcion de datos: route.params

A diferencia de otras pantallas que buscan datos en src/data/ con funciones como getUserById, esta pantalla recibe el objeto 'user' completo ya armado como parametro de navegacion, enviado por buscar_usuario_trainer.js cuando el entrenador toca un alumno en la lista de resultados. Es el mismo patron que usa ver_dia_x_user.js para recibir el objeto 'day' seleccionado: la pantalla anterior arma el objeto y lo pasa via navigation.navigate(pantalla, { user }).

```
const user = route?.params?.user;
```

El encadenamiento opcional (?) protege dos casos: que route.params no exista (navegacion sin parametros), o que exista route.params pero no tenga la clave 'user'. Ademas, cada campo individual que se muestra en pantalla tiene su propio fallback con el operador || '-' (por ejemplo user?.firstName || '-'), para no mostrar 'undefined' en pantalla si algun dato falta en el mock.

2. Estructura visual: 5 bloques de informacion

- **Perfil:** avatar circular con iniciales, nombre completo y email. Es el unico bloque sin el recuadro amarillo de borde, a proposito, para que combine visualmente con el header de perfil de mi_cuenta_user.js (mismo patron: avatar solido amarillo #FFC107 con texto oscuro adentro).
- **Datos Personales:** nombre, apellido, DNI y genero, en pares dentro de filas de 2 columnas.
- **Datos Fisicos:** edad, peso y altura, en una sola fila de 3 columnas.
- **Contacto y Salud:** telefono, telefono de emergencia, obra social y contraindicaciones medicas, apiladas verticalmente (una por fila), porque el contenido de texto es mas largo que en las otras secciones.
- **Entrenamiento:** objetivo principal del alumno, mostrado como un 'tag' con fondo oscuro ambar y texto amarillo (capsula/chip).

3. Patron row + box (columnas dinamicas)

Las secciones que muestran varios campos cortos lado a lado repiten el mismo patron de JSX/estilos, en vez de usar un componente extraido como ProfileDataCard (a diferencia de mi_cuenta_trainer.js / mi_cuenta_user.js). La razon es que la cantidad de columnas cambia entre secciones: 2 en Datos Personales, 3 en Datos Fisicos, 1 en Contacto y Salud - un componente generico tendria que aceptar una prop extra solo para eso, y no se justificaba la abstraccion para 3 usos distintos.

```
<View style={styles.row}>                {/* flexDirection: 'row', gap */}
  <View style={styles.box}>                {/* flex: 1, fondo gris #292929 */}
    <Text style={styles.label}>Nombre</Text>
    <Text style={styles.value}>{user?.firstName || '-'}</Text>
  </View>
  ...
</View>
```

Como 'box' tiene flex: 1, sin importar si hay 2 o 3 cajas en la fila, todas se reparten el ancho disponible en partes iguales automaticamente.

4. Estilo visual consistente con el resto de la app

- Todas las 'card' (salvo 'profile') usan el mismo lenguaje visual ya visto en otras pantallas: borderRadius: 16, borderWidth: 1.5, borderColor: '#FFE082' (borde amarillo palido).
- El titulo de cada seccion ('title') es amarillo fuerte #FFC107, igual que otros titulos de seccion en la app (por ejemplo el titulo de rutina en editar_rutina_trainer.js).
- El 'tag' del objetivo de entrenamiento reusa el mismo patron 'outline ambar' que los botones de editar_rutina_trainer.js: fondo oscuro #3A2B0D, texto amarillo #FFC107.

5. Responsividad

Como toda pantalla de la app, usa el hook useResponsiveLayout() para leer isLandscape y aplicar estilos alternativos (contentLandscape, headerLandscape) cuando el dispositivo esta en horizontal: el contenido se centra con un ancho maximo (maxWidth: 800) en vez de ocupar todo el ancho de la pantalla, siguiendo el mismo patron que las demas pantallas de la app.

6. Navegacion

El boton '<' del header no usa navigation.goBack(), sino que navega explicitamente de vuelta a 'buscar_usuario_trainer'. Esto asegura que el entrenador siempre vuelva a la lista de busqueda, sin importar la ruta exacta por la que llego a esta pantalla (evita quedar 'atascado' si en el futuro se agregan otras formas de llegar aca).